

RoadAssist Bharat

four-speaker run sheet

32 slides, four presenters, about 21 minutes plus questions. Each speaker owns whole modules, so an evaluator always knows who is accountable for what. Lines in the bordered blocks are meant to be said roughly as written — they are short on purpose.

SPEAKER 1

Problem & Module 1

Slides 1–8, 27–28

≈ 7 min 30 s

Name: _____

SPEAKER 2

Module 2 — Enabling Technology

Slides 9–13

≈ 4 min 15 s

Name: _____

SPEAKER 3

Module 3 — Infrastructure

Slides 14–18

≈ 4 min

Name: _____

SPEAKER 4

Module 4 — Architectures

Slides 19–26

≈ 5 min 30 s

Name: _____

SPEAKER 1

The problem, and what cloud computing is

Slides 1–8 · 6:00

01 HERO — ROADASSIST BHARAT

0:30

Good morning. We are presenting **RoadAssist Bharat** — an AI-assisted roadside assistance and emergency mobility platform for India.

It is a working system, not a mockup. And what we want to show you today is not just the application — it is how every layer of it maps onto Cloud Computing Modules 1 to 4.

Point at the two phones on the right — those are real screenshots of the running app, not artwork.

02 THE REAL-WORLD PROBLEM

0:50

When a vehicle breaks down, the mechanical fault is only the first problem.

The driver then cannot diagnose it. Cannot describe where they are on an unmarked road. Cannot tell who is nearby. And has no idea whether help is actually coming.

Every one of those six failures is an **information problem** before it is a mechanical one. And information problems are exactly what cloud platforms solve.

If asked for statistics — we deliberately did not invent any. The design is driven by that failure chain, not by numbers we could not source.

03 VISION — ONE PLATFORM

0:50

Six participants: the customer, AI diagnosis, the mechanic, the live map, the Raksha emergency system, and the road authority.

None of them talk to each other directly. Everything goes through the central platform — and that is what makes the platform **authoritative**.

If asked why — booking state is server-authoritative. If a mechanic's phone could tell a customer's phone that a job was finished, the two devices could disagree. The server is the single source of truth.

04 WHY ROADASSIST NEEDS THE CLOUD

0:45

Four kinds of consumer — customers, mechanics, authorities, and the public website — all reach one central platform, which draws on compute, storage, network and data.

Those six capabilities in the middle band are exactly the cloud characteristics from Module 1.

05 MODULE 1 — THE FIVE CHARACTERISTICS

1:10

Cloud computing has five essential characteristics, and each one is something concrete here.

On-demand self-service — a customer books assistance without anyone provisioning a thing.

Broad network access — it is a Progressive Web App, so any phone browser reaches it.

Resource pooling — one platform serves every customer and mechanic. Rapid elasticity — demand is not steady; monsoon and festival travel spike it. And measured service — usage is observable per request.

Strong line if challenged — "the weakest of the five in our build is rapid elasticity. We are designed for it but have not implemented automatic scaling, and we label that rather than claim it."

06 MODULE 1 — ROLES AND BOUNDARIES

1:00

Three layers. At the top, the cloud provider owns the physical estate. At the bottom, our end users — and there are seven distinct roles in our database.

In the middle sits RoadAssist, and this is the interesting part: we **consume** the provider's resources, but we ourselves **provide** a service to our users. A dual role.

07 MODULE 1 — DELIVERY & DEPLOYMENT MODELS

1:05

Infrastructure as a Service gives you virtual machines and you manage upward. Platform as a Service adds the managed runtime. Software as a Service is finished software over the network.

Look at the status column. Only SaaS is marked implemented — because that is genuinely what our users experience. The IaaS and PaaS rows are **architectural mapping**, not claims.

If asked which deployment model — hybrid. Public cloud for the citizen app for reach and cost; private for Raksha authority data and medical records, which is also where India's data-residency expectations point.

08 MODULE 1 — BENEFITS VS RISKS

0:50

Cloud brings scalability, availability, centralisation. It also brings real risks.

What I want you to notice is that we did not just list them — each risk has a concrete answer in the build. Network dependency is answered by an offline-first app with an idempotent sync journal. Data protection is answered by a hash-chained audit log where tampering is detectable .

HAND OFF → *"That is what the cloud is. [Name] will now take the technologies that make it possible."*

SPEAKER 2 Module 2 — cloud enabling technology

Slides 9–13 • 4:15

09 THE FIVE TECHNOLOGIES

0:50

Thank you. Module 2 covers the five technologies that make cloud computing possible: data centres, virtualization, web technology, multitenancy, and service technology.

I want to be precise about which of these we actually operate. Web and service technology are built into the application. Multitenancy exists in the database schema. Virtualization for us is containers , not a hypervisor we run. And the data centre belongs to whoever hosts it.

10 DATA CENTRE TECHNOLOGY

0:40

Everything we call "the cloud" ultimately runs on physical machines in a building with power and cooling.

A data centre supplies compute, storage, networking, power, cooling and monitoring. Those become cloud resources — and cloud resources are what RoadAssist services consume.

If asked why cooling matters to a software project — because it bounds server density, and therefore cost per unit of compute. That is ultimately what makes on-demand pricing possible for a consumer like us.

11 VIRTUALIZATION

1:00

Read this from the bottom. One physical server. Above it a hypervisor — the layer that lets one machine present itself as several isolated ones. Above that, virtual servers. And above those, the actual RoadAssist services.

Note the band at the top. We use OS-level virtualization — Docker containers — not a hypervisor we operate ourselves. That distinction matters and we state it.

Near-certain question — VM vs container — a virtual machine virtualizes hardware and runs its own kernel; a container virtualizes the operating system and shares the host kernel. Containers are lighter and start faster; VMs isolate more strongly.

12 WEB, MULTITENANT & SERVICE TECHNOLOGY

0:50

Web technology is fully implemented — 58 REST routes, and a Progressive Web App that installs to the home screen and keeps a 19-asset shell cached for offline use. Service technology too — eight independent capabilities behind one API contract.

Multitenancy I mark honestly as `schema present` : the fleet and government tenancy tables exist and queries are role-scoped, but we have not run separate tenants in production.

13 CONSUMING A CLOUD SERVICE — PAYMENTS

0:55

This is service technology in practice. Payments are not something we built — they are a cloud service we `consume` . Razorpay is the provider, we are the consumer.

The invoice total is the amount; it is never read from the request, so a client cannot choose what it owes. The browser gets only the key ID — the key `secret` never leaves our server. And the signature is re-computed on our side before anything settles.

The line worth landing — "there are two settlement paths, and the webhook is the one that matters. A customer can pay and immediately close the tab. The money has still moved. If settlement depended on their browser surviving, we would take real money and never close the invoice."

If asked "is this live?" — the integration is complete and verified by 22 automated tests against a stub. Live keys need a KYC-verified business account, so we do not claim live transactions.

HAND OFF → *"Those are the enabling technologies. [Name] will take the infrastructure mechanisms themselves."*

SPEAKER 3 Module 3 — cloud infrastructure mechanisms

Slides 14–18 · 4:00

14 THE FIVE MECHANISMS

0:40

Module 3 defines five infrastructure mechanisms: the network perimeter, virtual servers, cloud storage devices, cloud usage monitors, and resource replication.

I have marked each with what we actually have. Three are partial — real but limited. Two are architectural concepts we designed toward but did not build.

If asked why not just claim all five — because it would be false, and you can check. Precision about what exists is worth more than an inflated claim.

15 NETWORK PERIMETER

1:05

The perimeter is the controlled boundary between the untrusted internet and internal services. Ours is layered.

TLS terminates at the edge. Rate limits apply per phone number and per IP. Every request body is schema-validated. Then authentication, then a role gate — and then, crucially, an ownership check at the resource, not just at the route.

Best evidence in the whole talk — "we tested this by attacking it. A client that forges an X-Forwarded-For header to mint itself a fresh rate-limit bucket is ignored — every request lands in one bucket."

If asked why check at the resource — a route check asks "may this role do this kind of thing". A resource check asks "may this specific user touch this specific booking". Without the second, any mechanic could drive any customer's job.

16 VIRTUAL SERVER

0:40

A virtual server is an isolated, allocatable unit of compute carved out of physical infrastructure.

The important property is that our request handling is stateless — the session lives in a signed token, not in server memory. That is precisely what would allow more instances to be added without redesigning anything.

17 CLOUD STORAGE & REPLICATION

0:50

On the left is real: 61 tables covering identity, vehicles, bookings, money, emergencies, and a hash-chained audit log.

On the right is the replication architecture — a primary, replicas for read scale-out and failover, backups for point-in-time restore. Let me be explicit: today we run a single PostgreSQL instance. The dashed lines mean designed-for, not built.

If asked why not implemented — it is infrastructure configuration rather than an application change, and it needs a hosted database to be meaningful. The application would not need modifying.

18 CLOUD USAGE MONITOR

0:45

A usage monitor collects runtime data — CPU, memory, storage, network, requests, concurrency, errors. That data is what makes capacity planning and scaling decisions possible.

The bars here illustrate the mechanism; they are not our measurements. What we genuinely have is in the band below — a health endpoint reporting live database latency, per-request logging, and AI latency recorded per prediction.

HAND OFF → "With measurement in place, [Name] will show what architectures you can then build."

SPEAKER 4 Module 4 — architectures, and the whole system

Slides 19–26 • 5:30

19 THE EIGHT ARCHITECTURES

0:35

Module 4 defines the fundamental cloud architectures. Eight of them surround the platform here, plus cloud operations.

I will take these in pairs and show what each would mean for RoadAssist specifically.

If asked which you implemented — none as running infrastructure, and we say so. What we did implement are the properties that make them adoptable without a rewrite: stateless request handling, idempotency keys, server-authoritative state.

20 WORKLOAD DISTRIBUTION + RESOURCE POOLING

0:50

Our five request types have genuinely different shapes. Map tiles are bursty and cacheable. SOS is rare but must never wait.

Resource pooling means they all draw on one shared pool rather than each holding dedicated capacity. We have a small working example already — the map tile proxy keeps a shared cache, so a tile fetched for one user serves the next.

21 DYNAMIC SCALABILITY + ELASTIC CAPACITY

0:55

Demand here is not steady. Night is quiet. Monsoon brings poor roads and more breakdowns. Festival travel is the peak.

Dynamic scalability adds and removes whole instances. Elastic resource capacity resizes the instances you already have. Scaling out versus scaling up.

Near-certain question — the difference is *number* versus *size*. Out suits stateless web workloads; up suits a database that is hard to shard.

22 LOAD BALANCING + CLOUD BURSTING

0:50

A load balancer spreads requests and — just as importantly — health-checks them, so an unhealthy instance leaves rotation. It serves availability, not only throughput.

Cloud bursting is different. You run a baseline privately and overflow into public capacity at the limit, then release it. For us the trigger is a festival weekend — a short, fairly predictable burst.

23 ELASTIC DISK + REDUNDANT STORAGE

0:40

Elastic disk provisioning allocates storage as data arrives and bills on use, rather than buying a size up front that nobody can predict.

Redundant storage keeps a synchronised copy so a failure does not lose the data. For us the strongest case is the audit chain — it is only legally meaningful if it cannot be lost.

24 CLOUD OPERATIONS

0:45

Migration moves a workload between hosts. Static scheduling fixes allocation in advance. Dynamic scheduling recomputes it as demand changes — which is exactly why the usage monitor mattered.

And we have a real analogue: our dispatch engine is a scheduler. It queries live mechanic positions, ranks them, and allocates the job — a dynamic scheduling decision, just at application level.

25 THE COMPLETE ARCHITECTURE

0:55

This is the whole system in one picture. Four client surfaces. The API with 57 schema-validated routes. Eight platform services. Compute — marked partial, because it is containerised but not orchestrated. Storage. And at the base, the provider's infrastructure.

Down the foot are the cross-cutting concerns from Module 4 that wrap every layer.

Likely question — walk through one request — customer taps Request Assistance. HTTPS. Schema validation. Token verified, role checked, then vehicle ownership checked at the resource. State machine moves DRAFT to REQUESTED. Dispatch queries PostGIS for nearby mechanics and ranks them. Offers written. Every step logged with a request id.

26 THE COMPLETE JOURNEY

0:40

And this is what that architecture delivers. Login, vehicle, AI diagnosis, a guided request, mechanic matching, booking, live tracking, service, payment, review — and the review feeds back into the ranker that decides who gets dispatched next time.

This whole flow is covered by automated tests: 115 browser assertions driving both applications, plus 165 API, 26 security and 22 payment-gateway assertions.

HAND BACK → "[Name] will close."

SPEAKER 1 • RETURNS

Close

Slides 27–28 • 1:30

27 FINAL IMPACT

0:50

When the road stops, RoadAssist Bharat keeps you moving. AI, cloud, mobility and safety — four threads, one platform.

And throughout this presentation we have tried to be precise about which parts are built and which are architecture, because we think that distinction is what makes the cloud analysis worth anything.

0:40

Finally, this maps every syllabus topic to where it appears. Module 1 across slides 4 to 8. Module 2 across 9 to 13. Module 3, 14 to 18. Module 4, 19 to 24.

The point being that these are not an appendix — they are the spine of the presentation. RoadAssist was the worked example throughout.

Thank you. We are happy to take questions, and we can show the live system on any of these screens.

Who answers what

Agree this before you walk in. Nothing looks worse than four people looking at each other. If a question lands outside your area, say "[Name] built that" and hand over — that reads as a team, not as a gap.

TOPIC	OWNER	THE ONE-LINE ANSWER
What is cloud computing / characteristics	Speaker 1	On-demand network access to a shared pool of configurable resources, with minimal management effort.
IaaS / PaaS / SaaS · public vs private vs hybrid	Speaker 1	We are a SaaS application; IaaS and PaaS are how we would be hosted. Hybrid is the deployment we would choose.
Virtualization · VM vs container · multitenancy	Speaker 2	We use OS-level virtualization (containers), not a hypervisor. Multitenancy is in the schema, not yet in production.
Payments · Razorpay · why a webhook	Speaker 2	The browser is not a reliable narrator — a customer can pay and close the tab. The webhook is what guarantees settlement.
Security · perimeter · authorization · audit	Speaker 3	Layered: TLS, rate limits, schema validation, auth, role gate, ownership at the resource, hash-chained audit.
Storage · replication · monitoring	Speaker 3	PostgreSQL 16 + PostGIS, 61 tables. Replication is designed for, not built — single instance today.
Scaling · load balancing · bursting · scheduling	Speaker 4	None running. Stateless design and idempotency make them adoptable without a rewrite.
"Walk me through one request"	Speaker 4	Slide 25 is on screen — trace it with a finger, layer by layer.
"Show me it working"	Speaker 4	Open the live app. Have it already signed in on a second window.

The three answers that will earn you the most marks

1. **"What have you NOT implemented?"** — Automatic scaling, replication, load balancing and cloud bursting. We label every one of them as architecture rather than claiming them. Being able to answer this crisply is worth more than any feature.
2. **"How do you know it works?"** — 345 automated assertions across five suites, all passing: 17 unit, 165 API, 26 security, 115 browser, 22 payment gateway. We can run them here.
3. **"Is the AI real AI?"** — Be honest: it is a deterministic **rules engine**, and it is designed as the permanent fallback that any future model must beat. There is a provider interface for a real model. Do not call it machine learning.

If they cut you to 12 minutes

Drop slides, never drop the honesty badges — those are what the module marks hang on. Keep the same four speakers so the ownership story survives.

SPEAKER	KEEP	CUT	TIME
1	1, 2, 5, 7	3, 4, 6, 8	3:00
2	9, 11, 13	10, 12	2:30
3	14, 15, 17	16, 18	2:30
4	19, 21, 25, 26	20, 22, 23, 24	3:00
1	27, 28	—	1:00

Before you walk in

- ☐ Docker is running and the database is up — `npm run infra:up`
- ☐ The API is running — `npm start`, then check `localhost:4000/health` returns `ok`
- ☐ A second window is already signed in as the customer (`70000000000`) so "show me" costs you no time
- ☐ Presenter view is on — every slide has full speaker notes behind it
- ☐ Slide 13 and 15 are your strongest technical slides — Razorpay and the perimeter. Know them cold
- ☐ Agree the handoff names out loud once, beforehand. Reading "[Name]" off a script in the room is the one thing that will look unrehearsed
- ☐ Nobody says "we implemented Kubernetes / AWS / autoscaling." We did not, the deck does not claim it, and a single overclaim undoes the credibility the honesty badges buy you